LAB MANUAL
PART B

Roll No. C-60	**Name: Avdhut Madhavi**
Class. SE-C	**Batch: C2**
Date of Experiment:	**Date of Submission:**
Grade:	

B.1: Implement and trace the following programs. Write the code, compile/run it, and record the output after each task.

- Run the program with the items given in a different input order (e.g., reverse of the original order) and verify that the maximum value obtained remains the same.

```
#include <stdio.h>
#include <conio.h>

struct Item {
   int weight;
   int value;
   float ratio;
};

void sortItems(struct Item items[], int n) {
   int i, j;
   struct Item temp;

   for (i = 0; i < n - 1; i++) {
      for (j = 0; j < n - i - 1; j++) {
         if (items[j].ratio < items[j + 1].ratio) {
            temp = items[j];
            items[j] = items[j + 1];
            items[j + 1] = temp;
         }
      }
   }
}

float fractionalKnapsack(struct Item items[], int n, int W) {
   int i, remaining;
   float totalValue, fraction, addedValue;

   totalValue = 0.0;
   remaining = W;

   for (i = 0; i < n; i++) {
      items[i].ratio =
         (float)items[i].value / items[i].weight;
   }
```

```
    sortItems(items, n);

    printf("\nGreedy Selection Steps\n");
    printf("Weight Value Ratio Fraction Added Remaining\n");

    for (i = 0; i < n; i++) {
        if (remaining == 0)
            break;

        if (items[i].weight <= remaining) {
            totalValue = totalValue + items[i].value;
            remaining = remaining - items[i].weight;

            printf("%d %d %.2f Full(1.0) %d %d\n",
                items[i].weight,
                items[i].value,
                items[i].ratio,
                items[i].value,
                remaining);
        } else {
            fraction = (float)remaining / items[i].weight;
            addedValue = items[i].value * fraction;
            totalValue = totalValue + addedValue;

            printf("%d %d %.2f Partial(%.2f) %.2f 0\n",
                items[i].weight,
                items[i].value,
                items[i].ratio,
                fraction,
                addedValue);

            remaining = 0;
        }
    }

    return totalValue;
}

void displayItems(struct Item items[], int n) {
    int i;

    printf("\nItems (Weight, Value):\n");

    for (i = 0; i < n; i++) {
        printf("Item %d: (%d, %d)\n",
            i + 1, items[i].weight, items[i].value);
    }
}

int main() {
    struct Item items1[3] = {
        {10, 60, 0.0},
        {20, 100, 0.0},
        {30, 120, 0.0}
```

```
};

struct Item items2[3] = {
   {30, 120, 0.0},
   {20, 100, 0.0},
   {10, 60, 0.0}
};

struct Item items3[5] = {
   {5, 30, 0.0},
   {10, 40, 0.0},
   {15, 45, 0.0},
   {22, 77, 0.0},
   {25, 90, 0.0}
};

float maxValue;
int n;

clrscr();

printf("TEST CASE 1: Original Order\n");
n = 3;
displayItems(items1, n);
printf("Knapsack Capacity: 50\n");
maxValue = fractionalKnapsack(items1, n, 50);
printf("Maximum Total Value: %.2f\n", maxValue);

printf("\nTEST CASE 2: Reverse Order\n");
n = 3;
displayItems(items2, n);
printf("Knapsack Capacity: 50\n");
maxValue = fractionalKnapsack(items2, n, 50);
printf("Maximum Total Value: %.2f\n", maxValue);

printf("\nTEST CASE 3: Random Items\n");
n = 5;
displayItems(items3, n);
printf("Knapsack Capacity: 60\n");
maxValue = fractionalKnapsack(items3, n, 60);
printf("Maximum Total Value: %.2f\n", maxValue);

printf("\nCOMPLEXITY ANALYSIS\n");
printf("Sorting: O(n^2)\n");
printf("Greedy selection: O(n)\n");
printf("Overall time complexity: O(n^2)\n");
printf("Auxiliary space: O(1), excluding input storage\n");

printf("\nCHARACTERISTICS\n");
printf("1. Selects highest value/weight ratio first.\n");
printf("2. Optimal for Fractional Knapsack.\n");
printf("3. Not always optimal for 0/1 Knapsack.\n");

getch();
```

```
    return 0;
}
```

Output :



- **Do the analysis of fractional knapsack problem**
- The Fractional Knapsack problem is solved using the Greedy approach by selecting items with the highest value-to-weight ratio first.
- Items are sorted in descending order of their ratios before selection.
- If an item cannot fit completely, a fraction of it is selected to maximize the total value.
- The time complexity is $O(n^2)$using bubble sort, and the auxiliary space complexity is $O(1)$.
- The Greedy approach provides an optimal solution for the Fractional Knapsack problem.